

FUNCTION

BY SUJAL SHRESTHA

FUNCTION

In Python, a function is a reusable block of code that performs a specific task. Functions make the code modular, easier to read, and reduce redundancy.

In Python, functions can be broadly classified into two categories: inbuilt functions and user-defined functions.

A. Inbuilt Functions in Python

Inbuilt functions are pre-defined functions provided by Python to perform common tasks. These functions are always available and do not require explicit definition. Eg. `print()`, `len()`, `id()` and many more.

FUNCTION

B. User-Defined Functions

User-defined functions are created by programmers to perform specific tasks that may not be covered by inbuilt functions.

Defining a function

A function is defined using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`.

Example:

```
1 def greet():  
2     print("Hello World!")  
3 greet()  
4
```

```
Hello World!
```

```
=== Code Execution Successful ===
```

FUNCTION

Calling a User-Defined Function

To execute the code inside the function, use its name followed by parentheses, optionally passing arguments.

Example:

```
1 def name(name):  
2     print(f"My name is {name}")  
3 name("Sujal")  
4 name("Safal")
```

My name is Sujal

My name is Safal

=== Code Execution Successful

MATH FUNCTION

Python's math module provides mathematical functions and constants. These functions are useful for performing mathematical operations and computations. To use the math module, you need to import it first using `import math`.

Example:

```
1 import math as m
2 print("Square root:",m.sqrt(16))
3 print("Euler Number: ",m.e)
4 print("Pi value: ",m.pi)
5 print("Factorial: ",m.factorial(5))
6 print("Power: ",m.pow(2,3))
7
```

Square root: 4.0

Euler Number: 2.718281828459045

Pi value: 3.141592653589793

Factorial: 120

Power: 8.0

=== Code Execution Successful ===

PARAMETER VS ARGUMENT

The diagram illustrates the difference between function parameters and arguments. It shows a function definition and a function call with corresponding labels and brackets.

Function definition:

```
def my_sum(a, b):  
    return a + b
```

Annotations for the function definition:

- Function name:** Points to `my_sum` (green).
- Parameters:** A bracket groups `a` and `b` (purple).
- Function definition:** A bracket groups the entire `def my_sum(a, b): return a + b` block.

Function call:

```
my_sum(10, 20)
```

Annotations for the function call:

- Function call:** A bracket groups the entire `my_sum(10, 20)` expression.
- arguments:** A bracket groups `10` and `20` (red).

Parameters are mentioned in the function definition.
Actual parameters(arguments) are passed during a function call

QUESTIONS

1. Write a function that takes a name as a parameter and prints "Hello <name>".
2. Create a function that takes two numbers as parameters and prints their sum.
3. Write a function that takes a number as a parameter and prints its multiplication table.
4. Define a function that takes a string and prints its reverse.
5. Create a function that takes two numbers and prints the greater one.
6. Write a function that takes a number and prints its factorial.
7. Create a function that takes a username and password as parameters and prints "Login Successful" if they match predefined values, otherwise "Login Failed"

FUNCTION WITH ARGUMENT

Functions with arguments allow us to pass data into a function when calling it. This makes functions dynamic and capable of handling various inputs.

Types of Arguments in Python

1. Positional Arguments: Arguments are passed in order, and their position matters.

```
1 def info(name, address, age):  
2     print(f"Hello my name is {name}. I am {age}  
        years old. I live in {address}. ")  
3 info("Subash", "Jitpur", 26)
```

```
Hello my name is Subash. I am 26 years old. I live in  
Jitpur.
```

```
=== Code Execution Successful ===
```


FUNCTION WITH ARGUMENT

2.Keyword Arguments: Arguments are passed using key=value format, making the order irrelevant.

main.py Run <pre>1 def info(name, address, age): 2 print(f"Hello my name is {name}. I am {age} years old . I live in {address}. ") 3 info(age=35, address="Lalitpur", name="Safal Karki")</pre>	Output Clear <pre>Hello my name is Safal Karki. I am 35 years old. I live in Lalitpur. === Code Execution Successful ===</pre>
--	---

FUNCTION WITH ARGUMENT

3.Default Arguments: Arguments with predefined default values, used when no value is provided during the call.

main.py					Run	Output	Clear
1	def	info	(name, address="Kathmandu", age=31):	Hello my name is Anil Kumar Gupta. I am 31 years old. I live in			
2		print	(f"Hello my name is {name}. I am {age} years old	Kathmandu.			
			. I live in {address}. ")				
3	info	("Anil Kumar Gupta")	=== Code Execution Successful ===				

FUNCTION WITH ARGUMENT

4. Variable-Length Arguments: It is used when the number of arguments is not fixed.
- `*args`: For positional arguments (tuple).

main.py		Output
<pre>1 def data(*args): 2 print(args) 3 data(3, "TV", True, 1.5) 4</pre>		<pre>(3, 'TV', True, 1.5) === Code Execution Successful ===</pre>

FUNCTION WITH ARGUMENT

4. Variable-Length Arguments: It is used when the number of arguments is not fixed.
- `**kwargs`: For keyword arguments (dictionary).

main.py	Run	Output
<pre>1 def data(**kwargs): 2 print(kwargs) 3 data(number = 3, item = "TV", is_pruchase=True) 4</pre>		<pre>{'number': 3, 'item': 'TV', 'is_pruchase': True} === Code Execution Successful ===</pre>

QUESTIONS

- A. Write a program that accepts a variable length of arguments and print the value.
- B. Write a program to create a function named student() using the following conditions:
 - a. Program should accept the student's name and address and display both.
 - b. If the address is missing in the function call, then assign default value "Kathmandu".
- C. Write the following program to find sum of two numbers using a function.
- D. Write a function that takes a choice (1, 2, 3) and prints food items like:
 - 1 → Pizza
 - 2 → Burger
 - 3 → Mo:Mo
 - Other → Invalid choice

FUNCTION TYPES BASED ON PARAMETERS AND RETURN VALUES

Function with No Arguments and No Return Type

It is used for displaying a fixed message or performing tasks that do not require input/output.

```
def greet():  
    print("Hello, World!")
```

```
greet()
```

FUNCTION TYPES BASED ON PARAMETERS AND RETURN VALUES

Function with Arguments but No Return Type

Function with Arguments but No Return Type is used to process input and directly display the result.

```
def add_numbers(a, b):  
    print(a, b)
```

```
add_numbers(5, 7)
```

FUNCTION TYPES BASED ON PARAMETERS AND RETURN VALUES

Function with No Arguments but With Return Type

It is used when data is generated or calculated inside the function without external input.

```
def get_pi():  
    return 3.14159  
print(get_pi())
```


FUNCTION EXAMPLE

Function with Arguments and With Return Type

Return types function provide reusable results that can be stored, displayed, or fed into other functions. It uses input, performs operations, and gives results.

```
def add_numbers(a, b):  
    return a + b
```

```
result = add_numbers(5, 7)  
print(result)
```

QUESTION

1. Write a Python program to create the following:

a. Create an outer function that will accept two parameters, y and z.

b. Create an inner function inside an outer function that will calculate the addition of y and z.

c. Lastly, the outer function will add 5 into addition and return it

QUESTION

Problem: Write a function `count_vowels_and_consonants` that takes a string as input and returns the count of vowels and consonants in it.

Input: "Hello World"

Output: Vowels = 3, Consonants = 7

PASSING FUNCTION AS ARGUMENT

Passing a function as an argument means giving a function itself (not its return value) to another function so that the receiving function can decide when and how to execute it.

```
1 # Function to greet
2 def greet(name):
3     return f"Hello, {name}! Welcome."
4
5 # Higher-order function that accepts another
  function
6 def process(func, value):
7     return func(value) # func is executed here
8
9 # Example usage
10 print(process(greet, "Sujal"))
11 print(process(greet, "Anita"))
```

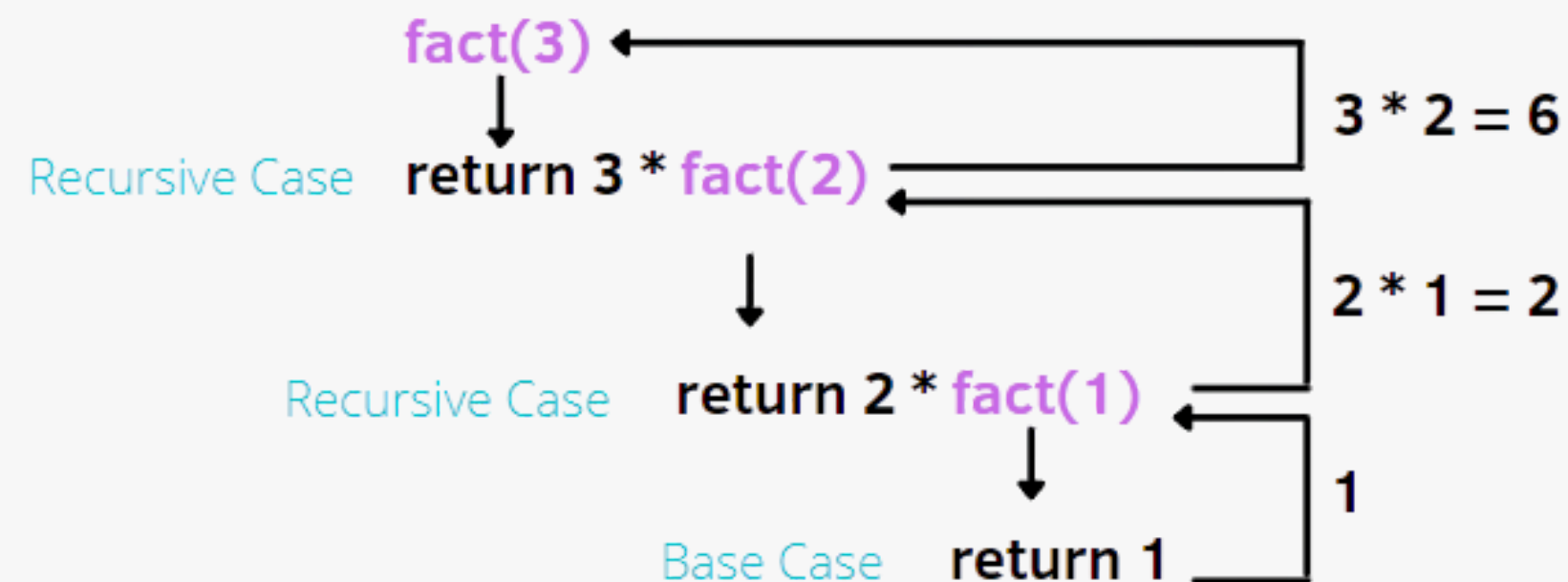
Hello, Sujal! Welcome.

Hello, Anita! Welcome.

=== Code Execution Successful ===

RECURSION

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller subproblems. Every recursive function must have a base case (to stop the recursion) and a recursive case (where the function calls itself). When a recursive function is executed, each call is placed on the call stack. The stack keeps track of the function calls until the base case is reached, then the results are returned back step by step as the stack unwinds.



RECURSION

Use cases of recursion:

- Mathematical problems like factorial, Fibonacci sequence, and computing greatest common divisor.
- Data structure traversal such as navigating trees, graphs, and linked lists.
- Divide and conquer algorithms including quicksort, mergesort, and binary search.
- Backtracking and exploration tasks like solving mazes, N-Queens, Sudoku, and directory traversal.

RECURSION

1	# Write a python program to countdown of	10
	a number using recursion	9
2	def countdown(n):	8
3	if n<1:	7
4	return	6
5	print(n)	5
6	countdown(n-1)	4
7	countdown(10)	3
		2
		1

RECURSION

```
1 # Write a python program to find the  
   factorial of a number using recursion  
2 def fact(n):  
3     if n==1 or n==0:  
4         return 1  
5     return n*fact(n-1)  
6 result=fact(5)  
7 print(f"The factorial of 5 is {result}.")
```

The factorial of 5 is 120.

=== Code Execution Successful ===

RECURSION

Questions:

1. Write a python program to display number 1-50 using recursion.
2. Write a python program to find the multiplication table of a number using recursion.
3. Write a python program to find the sum of first 10 natural numbers using recursion.

LAMBDA

A lambda function in Python is a small, anonymous function defined using the lambda keyword. Unlike normal functions created with def, a lambda function can be written in a single line and is often used for short, throwaway operations.

Syntax:

lambda arguments: expression

lambda → keyword to define the function

arguments → input parameters

expression → single expression whose result is returned

LAMBDA

A lambda function in Python is a small, anonymous function defined using the lambda keyword. Unlike normal functions created with def, a lambda function can be written in a single line and is often used for short, throwaway operations.

Syntax:

lambda arguments: expression

lambda → keyword to define the function

arguments → input parameters

expression → single expression whose result is returned

LAMBDA

```
1 # Lambda function to add two number
2 add=lambda a,b : a+b
3 print(add(5,5))
```

25

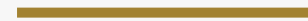
=== Code

```
6 # Lambda function to find the square of
  a number
7 square=lambda a : a **2
8 print(square(5))
```

25

=== Code

THANK YOU



FOR COMING

